

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG**



AN TOÀN VÀ BẢO MẬT THÔNG TIN

Đề tài
**ỨNG DỤNG RSA VÀ AES TRONG PHÁT TRIỂN HỆ THỐNG NHẮN
TIN BẢO MẬT**

Giảng viên hướng dẫn:
Thạc sĩ Phan Bích Chung

Sinh viên thực hiện:
B2207561 – Võ Duy Tân
B2207529 – Đào Thị Kim Khánh
B2207575 – Nguyễn Thị Thanh Tuyền

Cần Thơ, 10/2025.

LỜI CẢM ƠN

Chúng em xin gửi lời tri ân sâu sắc đến cô Phan Bích Chung, người đã tận tình hướng dẫn, chia sẻ kiến thức chuyên môn và đưa ra những góp ý quý giá, giúp định hình và hoàn thiện đề tài "Ứng Dụng RSA và AES Trong Phát Triển Hệ Thống Nhấn Tin Bảo Mật". Sự tận tâm và nhiệt huyết của cô là nguồn động lực lớn lao cho chúng em.

Chúng em xin chân thành cảm ơn các thầy cô trong Trường Công nghệ Thông tin và Truyền thông đã tạo điều kiện thuận lợi để chúng em triển khai và hoàn thành đề tài. Đặc biệt, chúng em trân trọng sự đồng hành, động viên và hỗ trợ không ngừng từ gia đình và bạn bè. Những lời khích lệ và ý kiến đóng góp từ các bạn đã tiếp thêm sức mạnh để chúng em vượt qua những thử thách trong suốt hành trình thực hiện.

Cuối cùng, chúng em xin bày tỏ lòng biết ơn đến các tác giả và những nguồn tài liệu tham khảo giá trị, đã cung cấp nền tảng kiến thức vững chắc, góp phần quan trọng vào sự thành công của đề tài.

MỤC LỤC

LỜI CẢM ƠN	i
MỤC LỤC	ii
DANH MỤC HÌNH	iii
ABSTRACT	iv
TÓM TẮT	v
I. PHẦN GIỚI THIỆU	1
1. Đặt vấn đề	1
2. Mục tiêu	1
3. Phạm vi nghiên cứu	1
II. PHẦN NỘI DUNG	2
1 Cơ sở lý thuyết	2
1.1 Cơ sở lý thuyết về bảo mật thông tin	2
1.2 Cơ sở lý thuyết về bảo mật thông tin	4
2 Phát triển hệ thống nhắn tin bảo mật	7
2.1 Kiến trúc tổng thể hệ thống	7
2.2 Quy trình bảo mật của hệ thống	11
2.3 Giao diện và tính năng	13
III. PHẦN KẾT LUẬN	14
4. Kết luận	14
5. Hướng phát triển	14
TÀI LIỆU THAM KHẢO	15

DANH MỤC HÌNH

1	Quy trình mã hóa và giải mã dữ liệu trong hệ thống bảo mật.	2
2	Minh họa hai mô hình mã hóa trong bảo mật thông tin.	3
3	Quy trình mã hóa và giải mã dữ liệu trong hệ thống bảo mật.	4
4	Minh họa hai mô hình mã hóa trong bảo mật thông tin.	5
5	Mô hình ba lớp Client–Server–Database của hệ thống.	8
6	Sơ đồ dữ liệu minh họa.	14

ABSTRACT

The project “Application of RSA and AES in Developing a Secure Messaging System” aims to build a secure messaging application operating in a local environment to meet the growing demand for information protection and ensure security in online communication.

The system is designed and developed using the Python programming language, combining two encryption algorithms: RSA for secure key exchange and AES for message encryption, ensuring end-to-end encryption. The application integrates a user-friendly interface through the tkinter library, supports encrypted chat history storage, and uses socket technology to establish connections between the server and clients. Experimental tests are conducted to evaluate the system’s resistance to common attacks such as eavesdropping and key spoofing, thereby confirming the system’s security and effectiveness.

The results show that the developed messaging application operates stably with a high level of security, effectively meeting the requirements for information safety in communication. The study also proposes future extensions, such as supporting multimedia data transmission, contributing to enhanced applicability and the further development of secure messaging systems in the digital era.

Từ khóa: Python, Cryptography, Encryption, RSA, AES, tkinter, End-to-End Encryption.

TÓM TẮT

Đề tài “Ứng dụng RSA và AES trong phát triển hệ thống nhắn tin bảo mật” nhằm xây dựng một ứng dụng nhắn tin an toàn hoạt động trong môi trường cục bộ, đáp ứng nhu cầu bảo vệ thông tin và đảm bảo an ninh trong giao tiếp trực tuyến.

Hệ thống được thiết kế và phát triển bằng ngôn ngữ Python, kết hợp hai thuật toán mã hóa RSA để trao đổi khóa an toàn và AES để mã hóa nội dung tin nhắn, giúp đảm bảo mã hóa đầu cuối (End-to-End Encryption). Ứng dụng tích hợp giao diện thân thiện với người dùng thông qua thư viện tkinter, hỗ trợ lưu trữ lịch sử trò chuyện dưới dạng mã hóa, và sử dụng socket để kết nối giữa server và client. Các thử nghiệm được tiến hành nhằm đánh giá khả năng chống lại các hình thức tấn công phổ biến như nghe lén và giả mạo khóa, từ đó khẳng định tính bảo mật và hiệu quả của hệ thống.

Kết quả đạt được là một ứng dụng nhắn tin hoạt động ổn định, bảo mật cao, đáp ứng tốt các tiêu chí về an toàn thông tin trong truyền thông. Đề tài cũng đề xuất hướng mở rộng trong tương lai như hỗ trợ truyền tải dữ liệu đa phương tiện, góp phần nâng cao tính ứng dụng và khả năng phát triển của các hệ thống nhắn tin bảo mật trong thời đại số hóa.

Từ khóa: Python, Cryptography, Encryption, RSA, AES, tkinter, End-to-End Encryption.

I. PHẦN GIỚI THIỆU

1. Đặt vấn đề

Trong thời đại số hóa hiện nay, giao tiếp trực tuyến đã trở thành một phần không thể thiếu trong cuộc sống hàng ngày, từ mạng xã hội, công cụ làm việc như Slack hay Microsoft Teams đến tin nhắn truyền thống trên điện thoại thông minh. Tuy nhiên, sự gia tăng của các mối đe dọa an ninh mạng, bao gồm tin tặc, tổ chức không được ủy quyền và các cuộc tấn công kiểu "người trung gian", đã đặt ra nhu cầu cấp thiết về việc bảo mật dữ liệu người dùng. Các nghiên cứu gần đây, như bài báo "Secure Chat Application with an End-to-End Encryption" (CIST 140), chỉ ra rằng mã hóa đầu cuối là giải pháp quan trọng để bảo vệ quyền riêng tư và toàn vẹn thông tin. Đề tài "Ứng dụng RSA và AES trong phát triển hệ thống nhắn tin bảo mật" được lựa chọn nhằm đáp ứng nhu cầu này, tận dụng sức mạnh của các thuật toán mã hóa tiên tiến để xây dựng một ứng dụng nhắn tin an toàn, hiệu quả và thân thiện với người dùng, đồng thời góp phần nâng cao nhận thức về bảo mật trong giao tiếp số.

2. Mục tiêu

Mục tiêu của đề tài này là:

- **Phát triển hệ thống nhắn tin với mã hóa đầu cuối:** Dựa trên mô hình của CIST 140, mục tiêu này tham khảo cách bài báo sử dụng RSA cho trao đổi khóa và AES cho mã hóa tin nhắn để đảm bảo chỉ người gửi và người nhận truy cập được nội dung [1].
- **Tích hợp RSA và AES:** Tham chiếu từ phương pháp luận của CIST 140, kết hợp RSA cho khóa bất đối xứng và AES cho khóa đối xứng, như được mô tả trong nghiên cứu của Sabah et al. (2005) [3].
- **Xây dựng GUI thân thiện:** Sử dụng tkinter như trong CIST 140 để tạo giao diện dễ sử dụng, tham khảo nguyên tắc cân bằng bảo mật và tiện ích từ Tariq et al. (2023) [4].
- **Kiểm tra bảo mật:** Tham khảo phương pháp kiểm tra bằng Wireshark trong CIST 140 để đánh giá hiệu quả mã hóa.
- **Mở rộng tính năng:** Lấy cảm hứng từ xu hướng đa phương tiện trong nghiên cứu của Chatterjee et al. (2018) [2] và kết luận của CIST 140 (Section 6) để đề xuất phát triển thêm cho tương lai.

3. Phạm vi nghiên cứu

Phạm vi nghiên cứu tập trung vào việc thiết kế và phát triển một ứng dụng nhắn tin bảo mật sử dụng Python, tích hợp mã hóa văn bản thông qua sự kết hợp của RSA và AES, kết nối qua socket, và xây dựng giao diện người dùng bằng thư viện tkinter, như được triển khai trong dự án. Tuy nhiên, đề tài có giới hạn ở việc chỉ hỗ trợ nhắn tin văn bản, không bao gồm các tính năng đa phương tiện như hình ảnh hoặc video, và chỉ triển khai trong môi trường cục bộ (localhost).

II. PHẦN NỘI DUNG

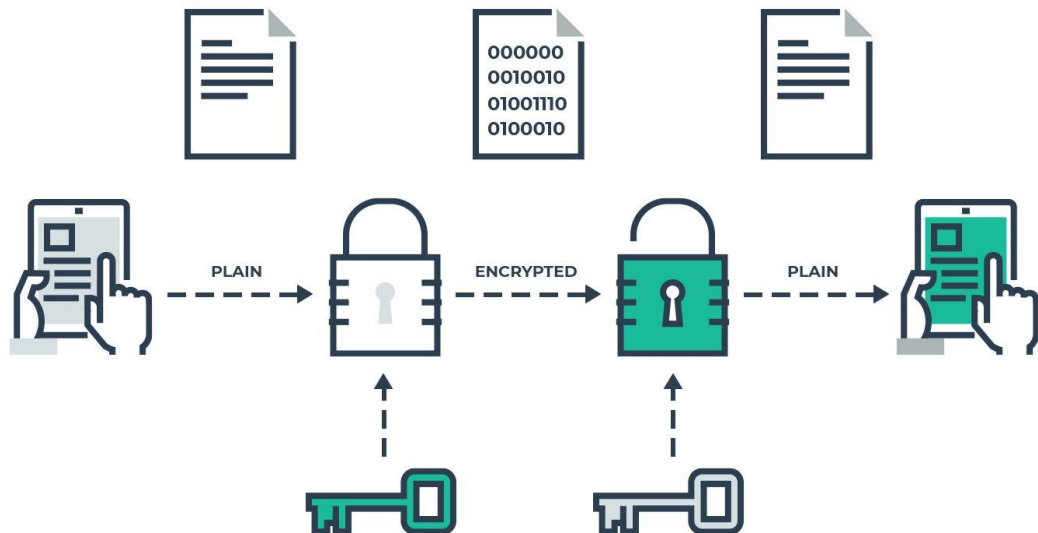
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

1.1 Cơ sở lý thuyết về bảo mật thông tin

1.1.1 Khái niệm mã hóa và giải mã

Mã hóa là quá trình chuyển đổi thông tin từ dạng có thể đọc hiểu (dữ liệu gốc) sang dạng biểu diễn không thể giải đoán hoặc nhận diện trực tiếp nếu thiếu khóa mã hóa phù hợp. Việc này giúp bảo vệ nội dung khỏi truy cập trái phép trong quá trình lưu trữ hoặc truyền đi, đảm bảo chỉ người có quyền mới đọc được dữ liệu thật.

Ngược lại, giải mã là quá trình khôi phục lại dữ liệu ban đầu từ dạng mã hóa. Người nhận sẽ dùng khóa giải mã tương ứng để mở khóa dữ liệu, từ đó đọc được thông tin gốc. Đây là bước cuối cùng để đảm bảo thông tin được truyền tải an toàn, chính xác và toàn vẹn giữa các bên.



Hình 1: Quy trình mã hóa và giải mã dữ liệu trong hệ thống bảo mật.

Hình 3 minh họa cho thấy dữ liệu ban đầu (plain text) được khóa lại thông qua quá trình mã hóa để tạo thành dữ liệu mã hóa (cipher text). Sau đó, bên nhận sử dụng khóa giải mã tương ứng để mở khóa và khôi phục lại dữ liệu gốc. Quá trình này đảm bảo rằng chỉ người có quyền truy cập hợp lệ mới có thể đọc được thông tin, từ đó bảo vệ tính bí mật và an toàn của dữ liệu trong suốt quá trình truyền tải.

1.1.2 Các yêu cầu cơ bản của hệ thống bảo mật thông tin

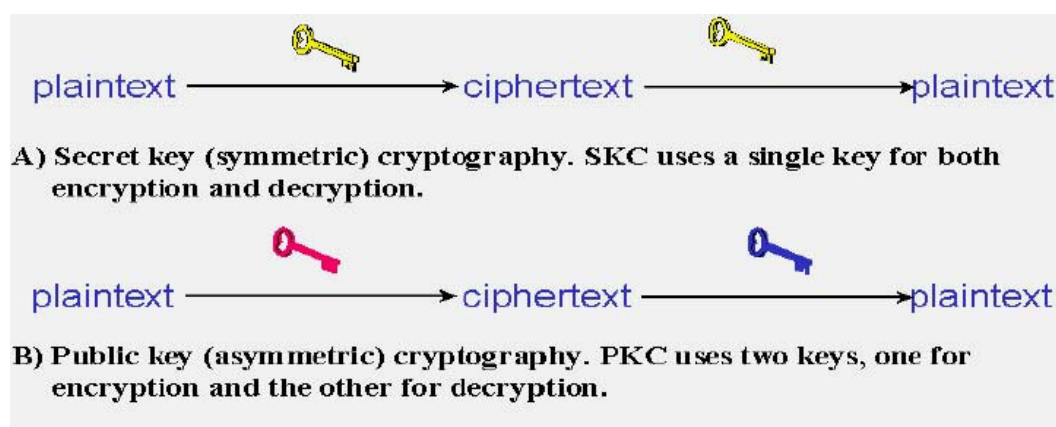
Một hệ thống bảo mật thông tin cần đáp ứng các yêu cầu cốt lõi sau:

- **Tính bí mật (Confidentiality):** Đảm bảo thông tin chỉ được truy cập bởi những người có quyền hợp lệ.
- **Tính toàn vẹn (Integrity):** Dữ liệu không bị thay đổi, giả mạo trong quá trình lưu trữ hoặc truyền tải.

- **Tính sẵn sàng (Availability):** Hệ thống luôn hoạt động ổn định, cho phép người dùng hợp lệ truy cập khi cần.
- **Tính xác thực (Authenticity):** Xác minh đúng danh tính người gửi, người nhận và nguồn gốc thông tin.
- **Tính chịu trách nhiệm (Accountability):** Đảm bảo mọi hành động trong hệ thống đều gắn với một danh tính xác định và có thể truy vết được.

1.1.3 Phân loại kỹ thuật mã hóa: đối xứng và bất đối xứng

Trong lĩnh vực bảo mật thông tin, kỹ thuật mã hóa được chia thành hai loại chính: mã hóa đối xứng và mã hóa bất đối xứng. Hai mô hình này khác nhau chủ yếu ở cách sử dụng khóa trong quá trình mã hóa và giải mã, như thể hiện trong Hình 4 dưới đây.



Hình 2: Minh họa hai mô hình mã hóa trong bảo mật thông tin.

Phần (A) biểu diễn mã hóa đối xứng (Symmetric Key Cryptography) – sử dụng cùng một khóa duy nhất cho cả mã hóa và giải mã. Phương pháp này có ưu điểm là tốc độ nhanh và dễ triển khai, nhưng vấn đề an toàn nảy sinh khi phân phối khóa cho các bên tham gia.

Phần (B) mô tả mã hóa bất đối xứng (Asymmetric Key Cryptography) – sử dụng hai khóa riêng biệt: khóa công khai (public key) để mã hóa và khóa bí mật (private key) để giải mã. Cách tiếp cận này giúp giải quyết khó khăn trong việc chia sẻ khóa, đồng thời tăng cường tính xác thực và an toàn cho hệ thống, dù tốc độ xử lý chậm hơn mã hóa đối xứng.

1.1.4 Khái quát về mô hình mã hóa đầu cuối

Mô hình mã hóa đầu cuối (End-to-End Encryption) là một phương pháp bảo mật dữ liệu, đảm bảo rằng thông tin được mã hóa tại thiết bị gửi và chỉ được giải mã tại thiết bị nhận, ngăn chặn bất kỳ bên thứ ba nào, kể cả nhà cung cấp dịch vụ, truy cập nội dung.

Cách hoạt động:

- Người gửi sử dụng khóa mã hóa (thường là khóa công khai của người nhận) để mã hóa dữ liệu ngay trên thiết bị người gửi.
- Dữ liệu mã hóa đi qua mạng, có thể qua máy chủ trung gian nhưng dưới dạng ciphertext và máy chủ không có quyền giải mã.

- Khi đến thiết bị người nhận, dữ liệu được giải mã bằng khóa bí mật tương ứng và chuyển thành nội dung đọc được.

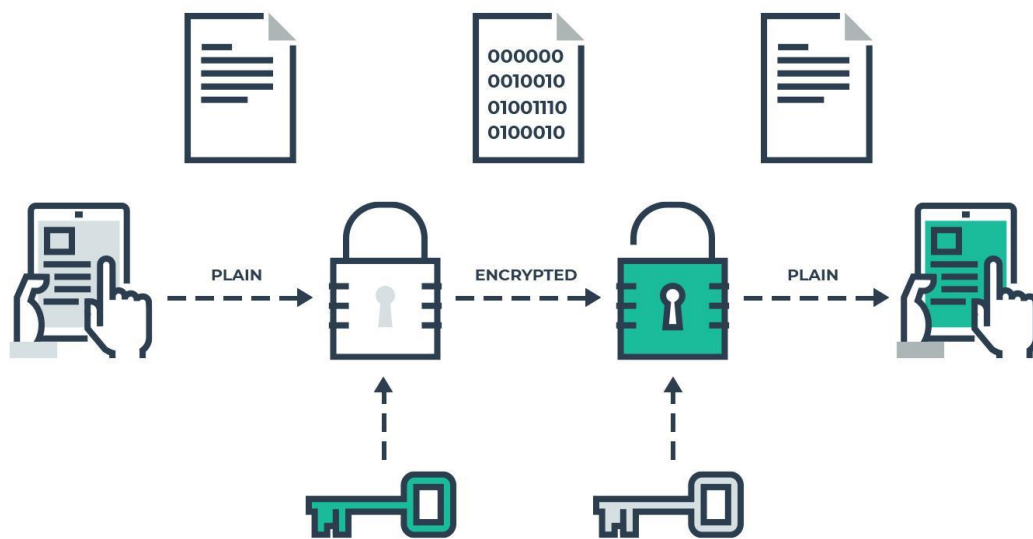
Mô hình mã hóa đầu cuối mang lại mức độ bảo mật cao. Tuy nhiên, mô hình này cũng có hạn chế như việc quản lý và phân phối khóa phức tạp, dễ phát sinh lỗi nếu người dùng không xác minh khóa đúng cách. Ngoài ra, nếu thiết bị đầu cuối bị tấn công hoặc nhiễm mã độc, nội dung vẫn có thể bị lộ dù quá trình truyền hoàn toàn an toàn.

1.2 Cơ sở lý thuyết về bảo mật thông tin

1.2.1 Khái niệm mã hóa và giải mã

Mã hóa là quá trình chuyển đổi thông tin từ dạng có thể đọc hiểu (dữ liệu gốc) sang dạng biểu diễn không thể giải đoán hoặc nhận diện trực tiếp nếu thiếu khóa mã hóa phù hợp. Việc này giúp bảo vệ nội dung khỏi truy cập trái phép trong quá trình lưu trữ hoặc truyền đi, đảm bảo chỉ người có quyền mới đọc được dữ liệu thật.

Ngược lại, giải mã là quá trình khôi phục lại dữ liệu ban đầu từ dạng mã hóa. Người nhận sẽ dùng khóa giải mã tương ứng để mở khóa dữ liệu, từ đó đọc được thông tin gốc. Đây là bước cuối cùng để đảm bảo thông tin được truyền tải an toàn, chính xác và toàn vẹn giữa các bên.



Hình 3: Quy trình mã hóa và giải mã dữ liệu trong hệ thống bảo mật.

Hình 3 minh họa cho thấy dữ liệu ban đầu (plain text) được khóa lại thông qua quá trình mã hóa để tạo thành dữ liệu mã hóa (cipher text). Sau đó, bên nhận sử dụng khóa giải mã tương ứng để mở khóa và khôi phục lại dữ liệu gốc. Quá trình này đảm bảo rằng chỉ người có quyền truy cập hợp lệ mới có thể đọc được thông tin, từ đó bảo vệ tính bí mật và an toàn của dữ liệu trong suốt quá trình truyền tải.

1.2.2 Các yêu cầu cơ bản của hệ thống bảo mật thông tin

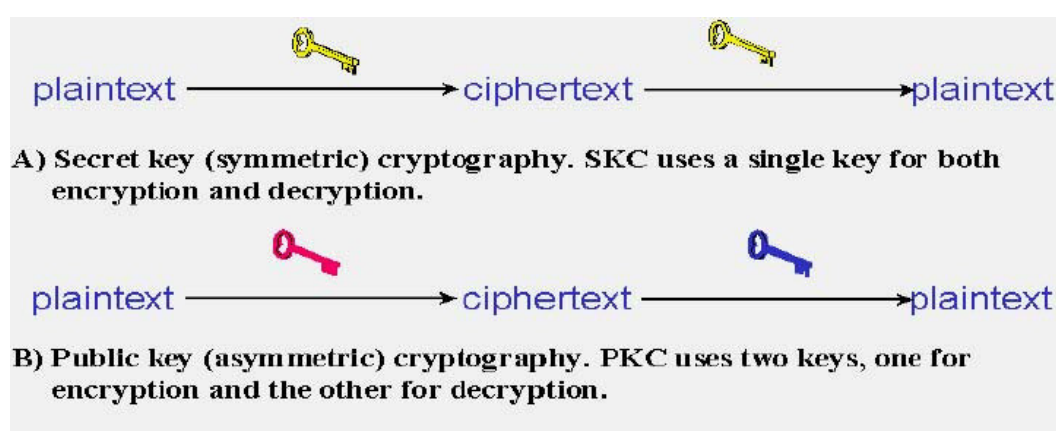
Một hệ thống bảo mật thông tin cần đáp ứng các yêu cầu cốt lõi sau:

- **Tính bí mật (Confidentiality):** Đảm bảo thông tin chỉ được truy cập bởi những người có quyền hợp lệ.

- **Tính toàn vẹn (Integrity):** Dữ liệu không bị thay đổi, giả mạo trong quá trình lưu trữ hoặc truyền tải.
- **Tính sẵn sàng (Availability):** Hệ thống luôn hoạt động ổn định, cho phép người dùng hợp lệ truy cập khi cần.
- **Tính xác thực (Authenticity):** Xác minh đúng danh tính người gửi, người nhận và nguồn gốc thông tin.
- **Tính chịu trách nhiệm (Accountability):** Đảm bảo mọi hành động trong hệ thống đều gắn với một danh tính xác định và có thể truy vết được.

1.2.3 Phân loại kỹ thuật mã hóa: đối xứng và bất đối xứng

Trong lĩnh vực bảo mật thông tin, kỹ thuật mã hóa được chia thành hai loại chính: mã hóa đối xứng và mã hóa bất đối xứng. Hai mô hình này khác nhau chủ yếu ở cách sử dụng khóa trong quá trình mã hóa và giải mã, như thể hiện trong Hình 4 dưới đây.



Hình 4: Minh họa hai mô hình mã hóa trong bảo mật thông tin.

Phần (A) biểu diễn mã hóa đối xứng (Symmetric Key Cryptography) – sử dụng cùng một khóa duy nhất cho cả mã hóa và giải mã. Phương pháp này có ưu điểm là tốc độ nhanh và dễ triển khai, nhưng vấn đề an toàn nảy sinh khi phân phối khóa cho các bên tham gia.

Phần (B) mô tả mã hóa bất đối xứng (Asymmetric Key Cryptography) – sử dụng hai khóa riêng biệt: khóa công khai (public key) để mã hóa và khóa bí mật (private key) để giải mã. Cách tiếp cận này giúp giải quyết khó khăn trong việc chia sẻ khóa, đồng thời tăng cường tính xác thực và an toàn cho hệ thống, dù tốc độ xử lý chậm hơn mã hóa đối xứng.

1.2.4 Khái quát về mô hình mã hóa đầu cuối

Mô hình mã hóa đầu cuối (End-to-End Encryption) là một phương pháp bảo mật dữ liệu, đảm bảo rằng thông tin được mã hóa tại thiết bị gửi và chỉ được giải mã tại thiết bị nhận, ngăn chặn bất kỳ bên thứ ba nào, kể cả nhà cung cấp dịch vụ, truy cập nội dung.

Cách hoạt động:

- Người gửi sử dụng khóa mã hóa (thường là khóa công khai của người nhận) để mã hóa dữ liệu ngay trên thiết bị người gửi.

-
- Dữ liệu mã hóa đi qua mạng, có thể qua máy chủ trung gian nhưng dưới dạng ciphertext và máy chủ không có quyền giải mã.
 - Khi đến thiết bị người nhận, dữ liệu được giải mã bằng khóa bí mật tương ứng và chuyển thành nội dung đọc được.

Mô hình mã hóa đầu cuối mang lại mức độ bảo mật cao. Tuy nhiên, mô hình này cũng có hạn chế như việc quản lý và phân phối khóa phức tạp, dễ phát sinh lỗi nếu người dùng không xác minh khóa đúng cách. Ngoài ra, nếu thiết bị đầu cuối bị tấn công hoặc nhiễm mã độc, nội dung vẫn có thể bị lộ dù quá trình truyền hoàn toàn an toàn.

CHƯƠNG 2. PHÁT TRIỂN HỆ THỐNG NHẮN TIN BẢO MẬT

2.1 Kiến trúc tổng thể hệ thống

2.1.1 Thành phần chính: Client – Server – Cơ sở dữ liệu

Hệ thống được thiết kế theo mô hình *client-server* ba lớp, trong đó mỗi thành phần đảm nhận vai trò riêng nhằm bảo đảm hiệu năng và tính bảo mật trong quá trình truyền tải thông tin.

Lớp Client: Ứng dụng được phát triển bằng *Flutter* (ngôn ngữ Dart), chịu trách nhiệm hiển thị giao diện trò chuyện, quản lý danh sách hội thoại, gửi và nhận tin nhắn, cùng các thao tác tải lên hoặc tải xuống dữ liệu. Tại lớp này, mọi hoạt động mã hóa và giải mã tin nhắn đều được xử lý hoàn toàn cục bộ nhằm đảm bảo nguyên tắc mã hóa đầu cuối (E2EE), qua đó ngăn máy chủ hoặc tin tặc truy cập vào nội dung tin nhắn.

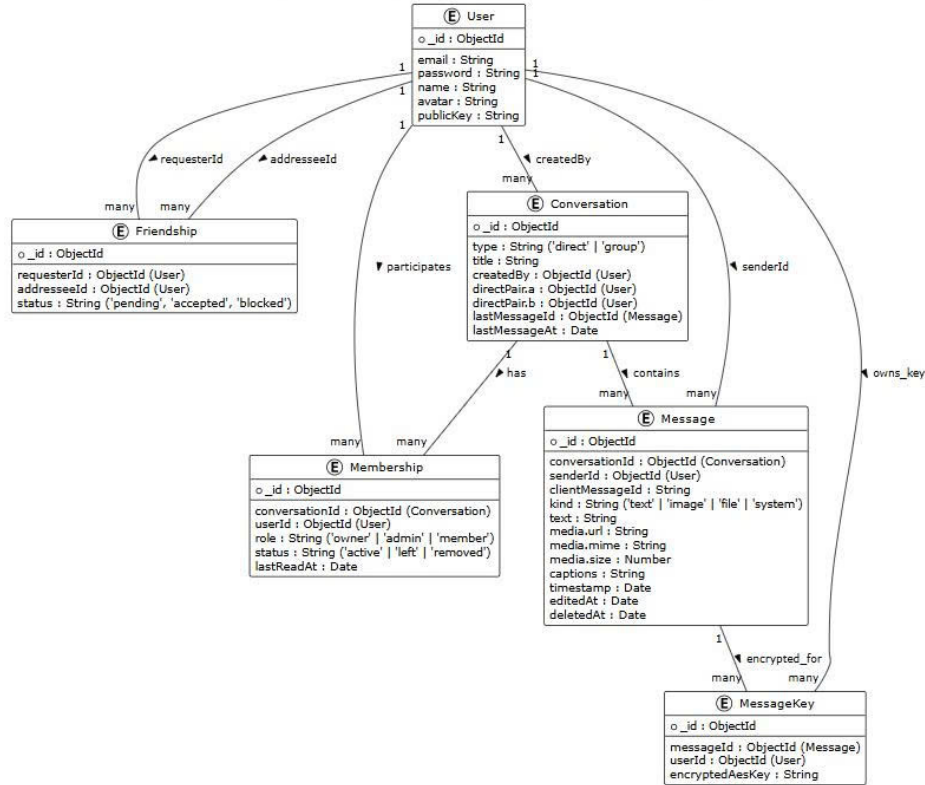
Ngoài các yêu cầu API thông thường, client duy trì kết nối *WebSocket* ổn định với máy chủ để truyền dữ liệu theo thời gian thực. Khi người dùng tham gia vào một cuộc trò chuyện cụ thể, client sẽ đăng ký vào một phòng tương ứng với mã định danh của cuộc trò chuyện. Mọi phiên client khác cùng tham gia hội thoại đó sẽ nhận được các sự kiện như tin nhắn mới hoặc lời mời kết bạn.

Lớp Server: Máy chủ được triển khai bằng *Node.js*, cung cấp *REST API* và *WebSocket Gateway*, đảm nhận vai trò trung gian trong việc truyền tải dữ liệu giữa các người dùng, xử lý xác thực, lưu trữ thông tin và các yêu cầu nghiệp vụ. Khi một tin nhắn mới được ghi nhận trong cơ sở dữ liệu, server thực hiện broadcast đến các client đang đăng ký cùng mã định danh của phòng chat, đảm bảo chỉ các thành viên hợp lệ của hội thoại đó mới nhận được dữ liệu. Trong toàn bộ quy trình, máy chủ không thực hiện giải mã nội dung tin nhắn mà chỉ lưu trữ bản mã và khóa AES đã được mã hóa bằng khóa công khai RSA, bảo đảm nguyên tắc mã hóa đầu cuối (E2EE).

Lớp Cơ sở dữ liệu: Hệ thống cơ sở dữ liệu được quản lý bằng *MongoDB*, nơi lưu trữ các thực thể chính như người dùng, quan hệ bạn bè, hội thoại, tin nhắn và khóa mã hóa. Việc kết hợp *MongoDB Change Streams* cho phép server lắng nghe các thay đổi dữ liệu và chuyển tiếp chúng qua *WebSocket*. Điều này giúp đảm bảo mọi hành động như gửi tin nhắn hay lời mời kết bạn được thể hiện tức thời trên toàn bộ thiết bị của người dùng.

Hình 5 minh họa kiến trúc tổng thể của hệ thống, trong đó client chịu trách nhiệm giao tiếp với người dùng, server xử lý nghiệp vụ và điều phối dữ liệu, còn cơ sở dữ liệu đảm bảo tính toàn vẹn và bảo mật thông tin.

Sơ đồ thực thể - liên kết hệ thống chat bảo mật



Hình 5: Mô hình ba lớp Client–Server–Database của hệ thống.

Bảng 1 mô tả chi tiết các trường dữ liệu của **bảng Conversation**, được sử dụng để lưu trữ thông tin về các cuộc trò chuyện trong hệ thống. Mỗi bản ghi trong bảng biểu thị một cuộc hội thoại, có thể là trò chuyện trực tiếp giữa hai người dùng hoặc trò chuyện nhóm có nhiều thành viên tham gia.

Bảng 1: Cấu trúc bảng dữ liệu Conversation

Trường	Kiểu dữ liệu	Mô tả
type	String	Loại cuộc trò chuyện: ‘direct’ hoặc ‘group’.
title	String	Tên nhóm (chỉ dùng khi là cuộc trò chuyện nhóm).
createdBy	Objectid (User)	Người tạo cuộc trò chuyện.
directPair.a	Objectid (User)	Người dùng A trong cuộc trò chuyện trực tiếp.
directPair.b	Objectid (User)	Người dùng B trong cuộc trò chuyện trực tiếp.
lastMessageId	Objectid (Message)	Tham chiếu đến tin nhắn gần nhất trong cuộc trò chuyện.
lastMessageAt	Date	Thời gian gửi tin nhắn cuối cùng.
timestamps	Date	Thời gian tạo và cập nhật tự động của bản ghi.

Bảng 2 mô tả **cấu trúc bảng dữ liệu Friendship**, được dùng để quản lý mối quan hệ bạn bè giữa các người dùng trong hệ thống. Mỗi bản ghi thể hiện một yêu cầu kết bạn cùng với trạng thái hiện tại của mối quan hệ.

Bảng 2: Cấu trúc bảng dữ liệu Friendship

Trường	Kiểu dữ liệu	Mô tả
requesterId	ObjectId (User)	Người gửi lời mời kết bạn.
addresseeId	ObjectId (User)	Người nhận lời mời kết bạn.
status	String	Trạng thái mối quan hệ: ‘pending’, ‘accepted’, ‘blocked’.
timestamps	Date	Thời gian tạo và cập nhật tự động.

Bảng 3 mô tả **cấu trúc bảng dữ liệu Membership**, được sử dụng để quản lý thông tin thành viên trong mỗi cuộc trò chuyện. Mỗi bản ghi thể hiện một người dùng tham gia vào một cuộc hội thoại cùng với vai trò, trạng thái và thời điểm tương tác gần nhất.

Bảng 3: Cấu trúc bảng dữ liệu Membership

Trường	Kiểu dữ liệu	Mô tả
conversationId	ObjectId (Conversation)	Cuộc trò chuyện mà người dùng tham gia.
userId	ObjectId (User)	Người dùng tham gia cuộc trò chuyện.
role	String	Vai trò trong nhóm: ‘owner’, ‘admin’, ‘member’.
status	String	Trạng thái tham gia: ‘active’, ‘left’, ‘removed’.
lastReadAt	Date	Thời điểm người dùng đọc tin nhắn cuối cùng.
timestamps	Date	Thời gian tạo và cập nhật tự động.

Bảng 4 trình bày **cấu trúc bảng dữ liệu Message**, nơi lưu trữ toàn bộ thông tin liên quan đến các tin nhắn trong cuộc trò chuyện. Mỗi bản ghi biểu thị một tin nhắn được gửi bởi người dùng, bao gồm nội dung văn bản, tệp đính kèm và các mốc thời gian tương ứng.

Bảng 4: Cấu trúc bảng dữ liệu Message

Trường	Kiểu dữ liệu	Mô tả
conversationId	ObjectId (Conversation)	Cuộc trò chuyện chứa tin nhắn.
senderId	ObjectId (User)	Người gửi tin nhắn.
clientMessageId	String	ID tạm thời của tin nhắn trên client.
kind	String	Loại tin nhắn: ‘text’, ‘image’, ‘file’, ‘system’.
text	String	Nội dung tin nhắn (được mã hóa AES).
media	Object	Thông tin tệp phương tiện: URL, MIME type, kích thước.
captions	String	Chú thích cho hình ảnh hoặc tệp đính kèm.
timestamp	Date	Thời gian gửi tin nhắn.
editedAt	Date	Thời điểm chỉnh sửa tin nhắn (nếu có).
deletedAt	Date	Thời điểm xóa tin nhắn.
timestamps	Date	Thời gian tạo và cập nhật tự động của bản ghi.

Bảng 5 mô tả **cấu trúc bảng dữ liệu MessageKey**, được sử dụng để lưu trữ khóa mã hóa AES của từng tin nhắn. Mỗi bản ghi thể hiện một khóa AES được mã hóa bằng RSA, gắn liền với người dùng sở hữu khóa và tin nhắn tương ứng.

Bảng 5: Cấu trúc bảng dữ liệu MessageKey

Trường	Kiểu dữ liệu	Mô tả
messageId	ObjectId (Message)	Tin nhắn tương ứng chứa khóa AES.
userId	ObjectId (User)	Người dùng sở hữu khóa AES đã mã hóa.
encryptedAesKey	String	Khóa AES được mã hóa bằng thuật toán RSA.
timestamps	Date	Thời gian tạo và cập nhật tự động của bản ghi.

Bảng 6 mô tả **cấu trúc bảng dữ liệu User**, được sử dụng để lưu trữ thông tin người dùng trong hệ thống. Mỗi bản ghi đại diện cho một tài khoản người dùng, bao gồm thông tin đăng nhập, dữ liệu nhận dạng và khóa công khai RSA dùng trong cơ chế mã hóa đầu cuối.

Bảng 6: Cấu trúc bảng dữ liệu User

Trường	Kiểu dữ liệu	Mô tả
email	String	Email của người dùng, đảm bảo duy nhất trong hệ thống.
password	String	Mật khẩu đã được mã hóa an toàn trước khi lưu trữ.
name	String	Tên hiển thị của người dùng trong các cuộc trò chuyện.
avatar	String	Đường dẫn ảnh đại diện của người dùng.
publicKey	String	Khóa công khai RSA được sử dụng để mã hóa tin nhắn gửi đến người dùng.
timestamps	Date	Thời gian tạo và cập nhật tự động của bản ghi.

Tổng thể, hệ thống cơ sở dữ liệu được thiết kế nhằm đảm bảo khả năng mở rộng, truy xuất hiệu quả và bảo mật thông tin người dùng. Các bảng dữ liệu được liên kết chặt chẽ với nhau: **User** quản lý danh tính và khóa công khai RSA, **Conversation** và **Membership** xác định cấu trúc các cuộc trò chuyện, trong khi **Message** và **MessageKey** lưu trữ nội dung tin nhắn cùng các khóa AES đã được mã hóa. Bảng **Friendship** giúp kiểm soát quan hệ bạn bè và quyền giao tiếp giữa các người dùng.

Mô hình tổ chức này không chỉ phục vụ cho hoạt động trao đổi dữ liệu theo thời gian thực mà còn bảo toàn nguyên tắc mã hóa đầu cuối. Nhờ đó, toàn bộ thông tin trò chuyện được bảo mật tuyệt đối — máy chủ chỉ lưu trữ bản mã và khóa được mã hóa, không thể truy cập vào nội dung gốc đảm bảo an toàn và tính riêng tư cho người dùng.

2.1.2 Nguyên tắc bảo mật giữa các thành phần

Hệ thống tuân thủ nghiêm ngặt các nguyên tắc bảo mật nền tảng nhằm bảo đảm tính **bí mật**, **toàn vẹn** và **xác thực** của thông tin trong suốt quá trình truyền và xử lý dữ liệu.

Xác thực và ủy quyền: Mọi người dùng đều phải trải qua quy trình xác thực trước khi truy cập hoặc gửi dữ liệu đến máy chủ. Cơ chế xác thực dựa trên mã thông báo (*token*) được cấp phát sau khi người dùng đăng nhập hợp lệ. Mỗi yêu cầu gửi đến *REST API* hoặc *WebSocket* đều phải kèm theo token hợp lệ để máy chủ tiến hành kiểm tra và xác nhận quyền truy cập. Cơ chế ủy quyền được áp dụng ở tầng ứng dụng, bảo đảm rằng người dùng chỉ có thể truy cập các hội thoại hoặc tài nguyên mà họ là thành viên hợp pháp.

Bảo mật kênh truyền: Dữ liệu trao đổi giữa *client* và *server* được bảo vệ thông qua các giao thức mã hóa lớp vận chuyển, cụ thể là HTTPS cho các yêu cầu REST và WSS (*WebSocket Secure*) cho các kết nối thời gian thực. Việc sử dụng các giao thức này giúp đảm bảo dữ liệu không thể bị nghe lén, đánh cắp hoặc giả mạo bởi các bên thứ ba trong

quá trình truyền qua mạng công cộng.

Mã hóa và quản lý khóa: Nội dung tin nhắn được mã hóa cục bộ tại *client* bằng thuật toán đối xứng AES. Khóa AES này sau đó được mã hóa bằng RSA — một cơ chế mã hóa bất đối xứng — sử dụng khóa công khai của người nhận để bảo vệ khóa phiên. Khi nhận tin nhắn, người nhận sử dụng khóa bí mật cá nhân của mình để giải mã khóa AES và khôi phục nội dung gốc. Máy chủ chỉ lưu trữ bản mã và khóa AES đã được mã hóa, hoàn toàn không có khả năng truy xuất nội dung gốc, qua đó duy trì mô hình mã hóa đầu cuối (E2EE).

2.1.3 Môi trường và công cụ phát triển

Công nghệ: Ứng dụng được phát triển bằng *Flutter/Dart* ở phía *client* và *Node.js/Express* ở phía *server*. Dữ liệu được lưu trữ trên *MongoDB*, trong đó các mô hình dữ liệu (*model*) bao gồm người dùng, quan hệ bạn bè, hội thoại, thành viên nhóm, tin nhắn và khóa mã hóa. Các thư viện mã hóa như *Pointycastle*, *Encrypt* và *Flutter Secure Storage* được tích hợp để đảm bảo xử lý an toàn các thao tác mã hóa **RSA–AES** cũng như quản lý khóa cục bộ trên thiết bị. Giao tiếp thời gian thực được thực hiện thông qua *WebSocket*, cho phép truyền và nhận sự kiện tức thời giữa các người dùng trong cùng một hội thoại.

Môi trường phát triển: Ứng dụng được xây dựng và kiểm thử trên môi trường giả lập (*Android Emulator*) và thiết bị thật, sử dụng *Flutter CLI*, *Gradle* và *Android Studio* để biên dịch. Phía máy chủ vận hành trong môi trường *Node.js* với khả năng đồng bộ dữ liệu thông qua *MongoDB Change Streams*, hỗ trợ cập nhật *real-time* khi có thay đổi tại tầng lưu trữ.

Cấu trúc hoạt động: Trong giai đoạn phát triển, *client* kết nối tới máy chủ thông qua địa chỉ cấu hình nội bộ. Khi triển khai chính thức, toàn bộ kênh truyền đều bắt buộc sử dụng HTTPS và WSS, kèm theo *token* xác thực hợp lệ cho mọi giao tiếp. Cấu trúc này giúp hệ thống vận hành ổn định, dễ dàng mở rộng và đảm bảo an toàn dữ liệu trong suốt vòng đời hoạt động.

2.2 Quy trình bảo mật của hệ thống

2.2.1 Tạo và lưu trữ cặp khóa RSA cho người dùng

Mỗi người dùng trong hệ thống đều được khởi tạo một cặp khóa **RSA** riêng biệt, bao gồm khóa công khai và khóa bí mật. Quá trình tạo khóa được thực hiện dựa trên bộ sinh số ngẫu nhiên an toàn, đảm bảo rằng mỗi cặp khóa là duy nhất và không thể dự đoán. Sau khi được sinh ra, hai khóa được chuyển sang định dạng PEM, định dạng tiêu chuẩn trong lưu trữ và trao đổi thông tin mật mã, giúp chúng có thể được chia sẻ hoặc sử dụng trong các bước mã hóa tiếp theo mà vẫn đảm bảo tính tương thích.

Khóa **công khai** được chia sẻ với những người khác trong cuộc trò chuyện, cho phép họ mã hóa khóa phiên khi gửi tin nhắn đến người sở hữu. Ngược lại, khóa **bí mật** chỉ được lưu trữ cục bộ trên thiết bị của người dùng, trong một vùng *Flutter Secure Storage*. Cơ chế lưu trữ này giúp ngăn chặn việc sao chép hoặc truy cập trái phép vào dữ liệu quan trọng, ngay cả khi thiết bị bị xâm nhập.

Nhờ quy trình trên, mỗi người dùng có một định danh mật mã độc lập, và toàn bộ hoạt động mã hóa – giải mã đều được thực hiện ở phía thiết bị. Điều này đảm bảo rằng không bên trung gian nào, kể cả máy chủ, có khả năng đọc hoặc giải mã nội dung tin nhắn, đúng theo nguyên tắc bảo mật của mô hình mã hóa đầu cuối.

2.2.2 Mã hóa tin nhắn bằng AES trước khi gửi

Trước khi một tin nhắn được gửi đi, hệ thống tiến hành mã hóa nội dung bằng thuật toán **AES** nhằm đảm bảo tính bí mật trong quá trình truyền tải. Khi người dùng nhập nội dung và nhấn gửi, ứng dụng sẽ sinh ra một khóa AES ngẫu nhiên cho từng tin nhắn. Khóa này được biểu diễn ở dạng Base64 để có thể truyền qua mạng và lưu trữ dễ dàng. Sau đó, hệ thống tạo *vector khởi tạo (IV)* ngẫu nhiên có độ dài 16 byte nhằm đảm bảo rằng cùng một nội dung nhưng được mã hóa nhiều lần vẫn cho ra kết quả hoàn toàn khác nhau, tránh việc suy luận mẫu dữ liệu.

Dữ liệu gốc (*plaintext*) được chuyển sang dạng byte và mã hóa theo chế độ **AES/CBC** với cơ chế *padding PKCS7*. Chế độ này giúp đảm bảo độ an toàn cao và tương thích với các chuẩn mã hóa hiện nay. Kết quả mã hóa là một chuỗi byte được ghép cùng IV rồi chuyển sang dạng Base64 trước khi gửi đi.

Quá trình này được thực hiện hoàn toàn cục bộ trên thiết bị của người gửi, không có dữ liệu bản rõ nào được gửi tới máy chủ. Sau khi mã hóa xong, nội dung tin nhắn (*ciphertext*) được đóng gói và gửi lên *backend* kèm theo danh sách khóa AES đã được mã hóa riêng cho từng người nhận.

Nhờ cơ chế này, toàn bộ nội dung trò chuyện đều được mã hóa trước khi rời khỏi thiết bị, đảm bảo rằng ngay cả máy chủ trung gian cũng không thể đọc hoặc giải mã tin nhắn của người dùng.

2.2.3 Trao đổi khóa AES qua kênh mã hóa RSA

Sau khi hoàn tất quá trình mã hóa nội dung tin nhắn bằng **AES**, ứng dụng cần gửi khóa AES này cho những người nhận trong cuộc trò chuyện để họ có thể tiến hành giải mã. Tuy nhiên, nếu gửi trực tiếp khóa AES thì sẽ tiềm ẩn rủi ro lộ thông tin. Vì vậy, hệ thống áp dụng cơ chế mã hóa bất đối xứng **RSA** để bảo vệ khóa phiên trước khi truyền đi.

Trước khi gửi tin nhắn, ứng dụng gửi yêu cầu đến máy chủ để lấy danh sách thành viên của cuộc trò chuyện, bao gồm khóa công khai **RSA** của từng người. Với khóa công khai này, ứng dụng tiến hành mã hóa khóa AES phiên riêng biệt cho từng thành viên. Cụ thể, cùng một khóa AES sẽ được mã hóa nhiều lần, mỗi lần bằng khóa công khai của một người nhận khác nhau.

Kết quả là mỗi người dùng sẽ nhận được một phiên bản khóa AES được mã hóa dành riêng cho họ, đảm bảo rằng chỉ thiết bị của họ mới có thể giải mã bằng khóa bí mật **RSA** lưu trữ cục bộ. Danh sách các khóa AES đã mã hóa được gửi kèm trong gói tin nhắn lên máy chủ, dưới dạng cặp `{userId, encryptedAesKey}`.

Trong toàn bộ quy trình này, máy chủ chỉ đóng vai trò trung gian truyền dữ liệu, hoàn toàn không có khả năng giải mã bất kỳ khóa AES nào vì không sở hữu khóa bí mật tương ứng. Cách triển khai này giúp đảm bảo khóa phiên chỉ có thể được giải mã tại phía người nhận, duy trì tính bảo mật tuyệt đối cho mô hình mã hóa đầu cuối (**E2EE**).

2.2.4 Giải mã tin nhắn ở phía người nhận

Khi có tin nhắn mới được gửi đến, ứng dụng phía người nhận sẽ nhận sự kiện "message" thông qua kênh *WebSocket*. Dữ liệu nhận được bao gồm nội dung tin nhắn đã được mã hóa bằng **AES**, cùng với danh sách các khóa AES đã được mã hóa riêng cho từng người trong cuộc trò chuyện.

Ngay khi nhận sự kiện, ứng dụng kiểm tra xem tin nhắn có thuộc cuộc trò chuyện hiện tại của người dùng hay không. Nếu đúng, hệ thống bắt đầu tiến trình giải mã. Đầu tiên, ứng dụng lấy khóa bí mật **RSA** của người dùng được lưu trữ an toàn trong vùng nhớ bảo mật của thiết bị (*Flutter Secure Storage*). Khóa này chỉ tồn tại cục bộ trên thiết bị người nhận và hoàn toàn không được máy chủ quản lý.

Tiếp theo, ứng dụng tìm trong danh sách `encryptedAesKeys` phần khóa AES được mã hóa dành riêng cho người nhận, được xác định thông qua trường `userId`. Khi tìm thấy, ứng dụng sử dụng khóa bí mật **RSA** để giải mã khóa AES phiên tương ứng. Khóa AES thu được sau khi giải mã sẽ được dùng để giải mã nội dung tin nhắn (*ciphertext*), khôi phục lại chuỗi văn bản gốc mà người gửi đã nhập.

Trong trường hợp hiếm khi người nhận chưa có khóa AES tương ứng, hệ thống sẽ gọi API phụ `/messages/key` để truy vấn lại khóa mã hóa từ máy chủ. Tuy nhiên, ngay cả trong tình huống này, chỉ thiết bị của người nhận mới có khả năng giải mã tin nhắn, vì chỉ nó mới sở hữu khóa bí mật **RSA** hợp lệ.

Kết quả cuối cùng là nội dung tin nhắn gốc được giải mã và hiển thị trong giao diện trò chuyện, hoàn tất vòng đời của một thông điệp được bảo vệ theo mô hình mã hóa đầu cuối (**E2EE**).

2.2.5 Bảo vệ khóa riêng tư và dữ liệu tin nhắn

Hệ thống áp dụng nhiều cơ chế bảo mật phối hợp để bảo vệ đồng thời khóa riêng tư của người dùng và dữ liệu trò chuyện trong suốt quá trình sử dụng.

Mỗi người dùng sau khi được khởi tạo cặp khóa **RSA** sẽ có khóa bí mật được lưu trữ an toàn trong vùng nhớ bảo mật riêng của thiết bị. Cơ chế này dựa trên hạ tầng bảo mật của hệ điều hành, đảm bảo rằng khóa không thể bị truy cập, sao chép hay đọc bởi bất kỳ ứng dụng hoặc tiến trình nào khác. Việc lưu trữ tách biệt này giúp ngăn ngừa nguy cơ rò rỉ khóa, ngay cả khi thiết bị bị tấn công ở cấp hệ thống tập tin.

Bên cạnh đó, toàn bộ quá trình mã hóa và giải mã đều diễn ra **cục bộ** trên thiết bị người dùng. Tin nhắn được mã hóa hoàn toàn trước khi rời khỏi máy và chỉ được giải mã khi đến đúng thiết bị của người nhận. Máy chủ chỉ đóng vai trò *lưu trữ* và *chuyển tiếp* dữ liệu đã mã hóa, không nắm giữ bất kỳ khóa bí mật nào và không thể đọc được nội dung trao đổi. Thiết kế này giúp duy trì mô hình mã hóa đầu cuối đúng nghĩa.

Hệ thống cũng tăng cường tính an toàn bằng cách sử dụng **khóa AES phiên riêng biệt cho từng tin nhắn**, giúp hạn chế rủi ro khi một khóa bị lộ. Nhờ đó, việc đánh cắp hoặc giải mã thành công một gói tin riêng lẻ sẽ không ảnh hưởng đến toàn bộ lịch sử trò chuyện.

Tổng thể, cơ chế này đảm bảo rằng khóa riêng tư của người dùng luôn được bảo vệ ở cấp thiết bị, và dữ liệu tin nhắn được bảo mật hoàn toàn cả trên đường truyền lẫn trong lưu trữ, phù hợp với các nguyên tắc bảo mật của hệ thống mã hóa đầu cuối hiện đại.

2.3 Giao diện và tính năng

III. PHẦN KẾT LUẬN

4. Kết luận

Mô tả use case chính.

5. Hướng phát triển

Chèn hình minh hoạ Hình 6.

Example figure placeholder
(figures/example.png)

Hình 6: Sơ đồ dữ liệu minh hoạ.

TÀI LIỆU THAM KHẢO

- [1] H. Carranza **and** et al., “Secure Chat Application with an End-to-End Encryption,” *in Proceedings of the 10th World Congress on Electrical Engineering and Computer Systems and Sciences (EECSS'24)* 2024.
- [2] N. Chatterjee **and** et al., “Real-time Communication Application Based on Android Using Google Firebase,” *International Journal of Advance Research in Computer Science and Management Studies*, 2018.
- [3] N. Sabah **and** et al., “Developing an End-to-End Secure Chat Application,” 2005.
- [4] U. Tariq **and** et al., “A Critical Cybersecurity Analysis and Future Research Directions for the Internet of Things,” *Sensors*, 2023.